<u>User Guide- Tic Tac Toe</u>

This project is a simple two-player Tic-Tac-Toe game developed in Python and designed to be played in the command-line interface. It allows two human players to take turns marking spaces on a 3x3 grid until one player wins or the game ends in a draw. The code emphasizes simplicity, clarity, and basic game logic, making it a great example for beginner programmers or those learning Python.

To use the program, simply paste the Python code into a code cell in Google Colab and run it. The game will start in the output area, where you can interact with it using text input. Once the game starts, Player X goes first, followed by Player O. Each player is prompted to enter their move using a coordinate format like “1 2,” which represents the row and column on the board. The game board updates after each turn, and the program checks for a win condition after every move. If a player fills a full row, column, or diagonal with their symbol, the game announces the winner and ends. If the board becomes full with no winner, the game ends in a draw.

The code is structured with several key functions. The print_board() function displays the current state of the board with clear separators. The check_winner() function evaluates all possible winning combinations to determine if the current player has won. The is_full() function checks if the board is full, which is used to determine a draw. The get_move() function handles user input, ensuring it is valid, within range, and that the chosen cell is not already occupied. The main play_game() function brings all these components together in a loop that runs until there is a winner or a draw.

This program was developed using only built-in Python functionality, with a focus on readability and logic. No external libraries are required, and the game is fully self-contained. It was designed to demonstrate basic programming skills such as loops, conditionals, user input handling, and simple list manipulation.

There are a few known limitations. The game currently supports only two local human players and does not feature a single-player mode against a computer or AI. There is no graphical user interface; it runs entirely in the terminal. Input handling is limited to catching basic formatting errors, and players must restart the program to play again after the game ends. There is also no scorekeeping between rounds.

In future versions, several enhancements could be made to improve the user experience. Adding an AI opponent would allow for single-player mode, while implementing a GUI with a library like Tkinter or Pygame could make the game more interactive. Additional features such as a restart option, score tracking, or move history would also increase its functionality and polish.

Overall, this project demonstrates the core mechanics of a turn-based game and provides a solid foundation for further development in Python.